

familydoctor포트폴리오

1. 프로젝트 개요

항목	내용
프로젝트명	FamilyDoctor - AI 주치의
유형	개인 프로젝트
기간	2026년
배포 URL	https://familydoctor-sc6ue6cf3q-uc.a.run.app
저장소	https://github.com/c0z0c/familydoctor

가족 구성원의 건강 히스토리를 누적 관리하고, Gemini LLM이 오케스트레이터로 대화 흐름을 조절하며 MedGemma 4B GGUF 모델이 로컬에서 직접 의료 조언을 생성하는 웹 애플리케이션.

2. 기술적 도전 과제와 해결

2.1 로컬 LLM(GGUF)과 클라우드 서비스의 통합

문제: Google Cloud Run은 stateless 컨테이너 환경으로 2.5GB GGUF 모델을 매번 로드해야 한다. 모델 파일을 이미지에 포함하면 빌드/배포 시간이 급증하고, 환경 변수나 DB 연결처럼 일반적인 "준비 완료" 판단이 불가능하다.

해결:

GCS 버킷 → GCS FUSE → /mnt/models (Cloud Run gen2 볼륨 마운트)
└─ 이미지 크기 유지 + 모델 파일 분리 관리

- MedGemmaService를 별도 자식 프로세스(multiprocessing.spawn)로 분리하여 FastAPI 메인 프로세스와 메모리 공간 격리
- 부모-자식 간 통신을 multiprocessing.Queue로 처리, 추론 요청/응답을 비동기로 위임
- prepare_async()를 asyncio.create_task()로 백그라운드 실행하여 서버는 즉시 기동

2.2 Cloud Run cold start에서의 503 문제

문제: Cloud Run 기본 TCP probe는 uvicorn 포트가 열리면 즉시 Ready로 판단한다. GGUF 모델 예열(~41초)이 완료되기 전에 트래픽이 유입되어 /api/medical/chat이 503을 반환하는 상황이 반복됐다.

실측 데이터 (Cloud Run 로그, revision familydoctor-00014-wg7):

단계	시각	경과
워커 프로세스 기동 + 예열 대기 시작	08:22:39	0초
llama_context 출력 (모델 파일 읽기 완료)	08:23:16	+36.7초 (90%)
예열 추론 완료 (16토큰)	08:23:20	+40.6초 합계
/health 200 (startup probe 통과)	08:23:20	워커 완료와 동시
Cloud Run revision Ready	08:23:20	컨테이너 시작부터 1분 12초

해결:

1. /health 엔드포인트가 model_ready=False일 때 HTTP 503 반환

2. cloudbuild.yaml에 startup probe 추가

```
--startup-probe=httpGet.path=/health,httpGet.port=8080,  
initialDelaySeconds=10,periodSeconds=10,  
failureThreshold=90,timeoutSeconds=5
```

3. startup probe가 /health 200을 확인한 후에만 트래픽 전환 → 503 완전 차단

2.3 SIGTERM 경합 조건 버그

문제: Cloud Run이 인스턴스 교체 시 SIGTERM을 보내면 `_on_sigterm` 핸들러가 `model_ready=False`로 설정한다. 그러나 백그라운드 예열 Task가 이후에 완료되면서 `model_ready=True`로 되돌려 SIGTERM을 받은 인스턴스가 다시 트래픽을 받는 문제가 발생했다.

해결:

```
# MedGemmaService  
self._sigterm_received: bool = False  
  
# main.py _on_sigterm 핸들러  
medgemma._sigterm_received = True  
medgemma.model_ready = False  
  
# prepare_async() 완료 직전 가드  
if self._sigterm_received:  
    logger.warning("SIGTERM 수신 상태 - model_ready=True 설정 건너뛸")  
    return  
self.model_ready = True
```

2.4 LangGraph ReAct Agent Tool 설계

문제: 단순 Gemini API 직접 호출(GeminiService 레거시)로는 대화 흐름 제어, 로그 저장, MedGemma 호출을 하나의 요청 안에서 일관성 있게 처리하기 어려웠다.

해결: LangGraph `create_react_agent`로 전환하여 5개 Tool을 클로저 바인딩:

Tool	역할
<code>clarify_user</code>	PQRST 문진 프로토콜 기반 추가 질문
<code>check_outcome</code>	미완료 로그 경과 확인
<code>consult_medgemma</code>	MedGemmaService 로컬 추론 호출
<code>save_log</code>	증상 로그 Firestore 저장
<code>complete_log</code>	미완료 로그 완료 처리

Gemini LLM(`gemini-3.1-flash-lite-preview`)이 PQRST 문진 가이드와 중복 질문 금지 규칙이 포함된 시스템 프롬프트를 기반으로 Tool을 자율 선택한다.

2.5 비용 최소화 설계

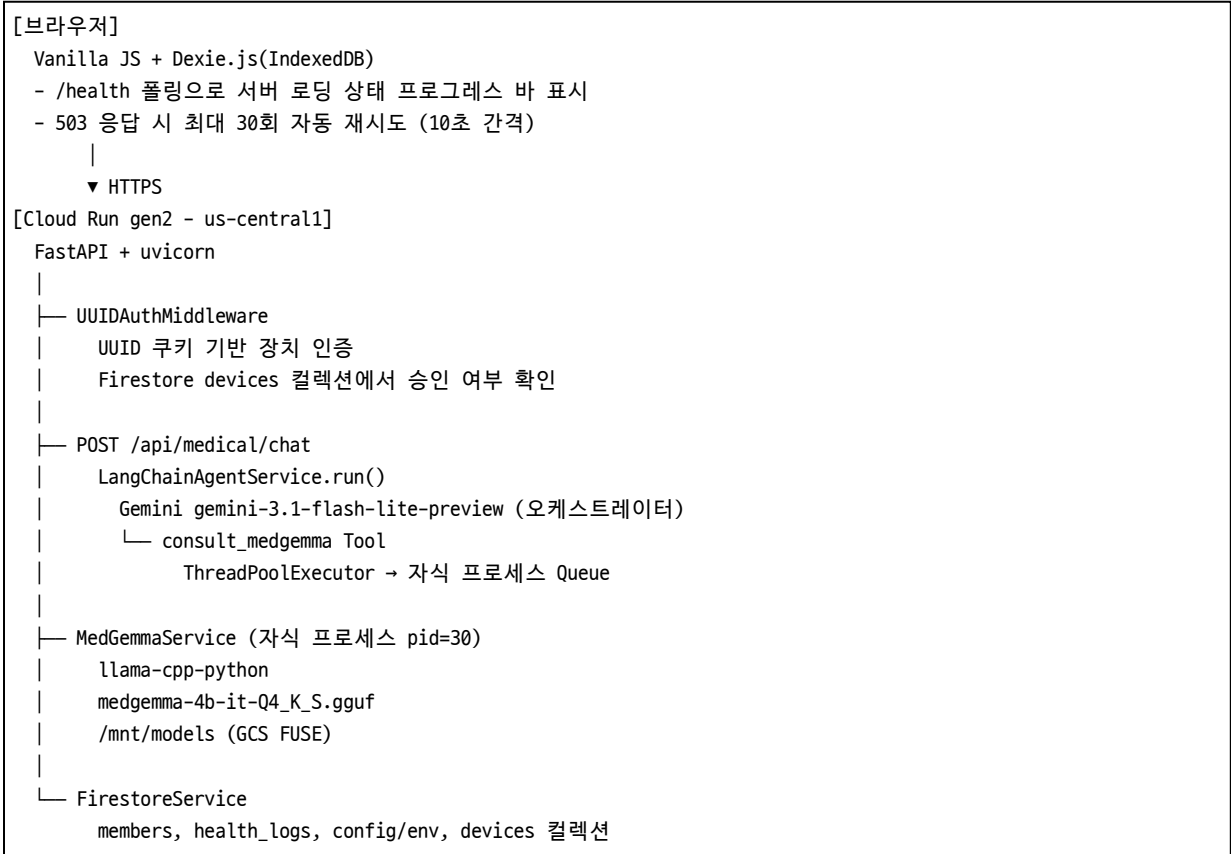
월 1~2회 호출하는 개인 프로젝트 특성상 비용 최소화가 핵심 요구사항이었다.

설정	이전	이후	이유
<code>min-instances</code>	0	0	idle 비용 없음 유지
<code>max-instances</code>	3	1	동시 사용자 없음

concurrency	10	1	단일 사용자
cpu-boost	활성	비활성	cold start 중 CPU 2배 과금 제거

예상 월 비용:
0.10 미만 (GCS 스토리지 0.05 포함)

3. 시스템 아키텍처



4. 핵심 구현 코드
4.1 MedGemmaService 멀티프로세스 구조

```

# 자식 프로세스에서 모델 1회 로드 후 Queue로 요청 수신
def _worker_loop(req_queue, res_queue, model_path):
    llm = Llama(model_path=model_path, n_ctx=4096, n_threads=1)
    while True:
        req = req_queue.get()
        if req is None: # 종료 신호
            break
        raw = llm.create_chat_completion(messages=req["messages"], ...)
        res_queue.put({"ok": True, "result": raw["choices"][0]["message"]["content"]})

# 부모 프로세스(FastAPI)에서 비동기 위임
async def prepare_async(self):
    await loop.run_in_executor(self._executor, self._ensure_worker)
    await loop.run_in_executor(self._executor, self._warmup) # 실측 40.6초
    if not self._sigterm_received:
        self.model_ready = True

```

4.2 /health startup probe 연동

```

@app.get("/health")
async def health_check(request: Request):
    medgemma = request.app.state.medgemma
    body = {
        "status": "healthy" if medgemma.model_ready else "initializing",
        "model_ready": medgemma.model_ready,
        "model_downloading": medgemma.model_downloading,
    }
    if not medgemma.model_ready:
        return JSONResponse(content=body, status_code=503)
    return body

```

4.3 프론트엔드 서버 로딩 폴링

```

async function waitForServer() {
    if (await checkServerReady()) return; // 이미 warm이면 즉시 통과

    overlay.classList.remove("hidden"); // 프로그레스 바 오버레이 표시
    // 1초마다 경과 시간 갱신, 10초마다 /health 폴링
    // model_ready=true 되면 오버레이 자동 제거
}

```

5. 개발 회고

잘 된 점

- GCS FUSE 볼륨 마운트로 이미지 크기와 모델 파일 관리 분리 성공
- startup probe 도입으로 cold start 중 503 문제 완전 해결

- SIGTERM 경합 조건을 _sigterm_received 플래그로 명확하게 가드
- LangGraph ReAct Agent 전환으로 대화 흐름 제어 코드가 선언적으로 단순화

개선 여지

- GCS FUSE 네트워크 읽기로 인한 cold start 5~6분은 모델 파일을 Docker 이미지에 포함하면 30~60초로 단축 가능 (Artifact Registry 비용 증가 트레이드오프)
- n_ctx=4096으로 제한 중 (n_ctx_train=131072). 메모리 확보 시 확장 가능
- n_threads=1로 단일 스레드 추론 중. CPU 2개 활용을 위해 조정 가능
- 개인 프로젝트로 비용은 최소화하였으나 더 빠른 응답이 필요하다면 항상 켜진 인스턴스(min-instances=1)로 변경 고려 가능

6. 기술 스택 요약

Language	: Python 3.12
Framework	: FastAPI, LangChain, LangGraph
LLM	: Google Gemini gemini-3.1-flash-lite-preview (오케스트레이터) MedGemma 4B GGUF Q4_K_S (로컬 추론, llama-cpp-python)
Database	: Google Cloud Firestore
Frontend	: Vanilla JS, Dexie.js, marked.js
Infra	: Google Cloud Run gen2, GCS FUSE, Cloud Build, Artifact Registry
Auth	: UUID 기반 장치 인증 (Firestore devices 컬렉션)